Ders 11 Genişletilmiş Notu: Test Spesifikasyonu, Risk Yönetimi, Sürüm Yönetimi ve Mobil Uygulama Testi

Ders 11 Genişletilmiş Notu: Test Spesifikasyonu, Risk Yönetimi, Sürüm Yönetimi ve Mobil Uygulama Testi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Gereksinim Belgesi (SRS): Fonksiyonel ve Fonksiyonel Olmayan Gereksinimler
- 2. Bir Yazılımın Test Planı ve Test Raporunda Neler Olmalı?
- 3. Test Yönetimi ve Test Spesifikasyonu
- 4. Test Ekibi, Bağımsızlık ve Optimum Test Miktarı
- 5. Teslimden Sonra Kalite: Yazılım Bakımı
- 6. Konfigürasyon Yönetimi
- 7. Yazılım Risk Yönetimi
- 8. Yeniden Kullanılabilirliği Artıran Yaklaşımlar
- 9. Sürüm Yönetimi (Release Management)
- 10. Mobil Uygulamalarda Kalite ve Test
- Örnek Sorular

1. Gereksinim Belgesi (SRS): Fonksiyonel ve Fonksiyonel Olmayan Gereksinimler

Ders, grupların daha önce teslim ettiği gereksinim belgelerinin (SRS, Software Requirements Specification) değerlendirilmesiyle başladı. Hoca belgeleri tek tek gözden geçirirken, iyi bir SRS belgesinin yapısını da hatırlattı. Bazı grupların projeyi önceki derslerden (yazılım mühendisliği, sistem analizi) aldığını, bunun sorun olmadığını belirtti. Örneğin bir grubun konusu otopark otomasyonuydu.

SRS formal bir rapordur. Bu yüzden baş kısmında bazı tanımlayıcı bilgiler yer alır: Bu belge nasıl bir projenin gereksinimlerini veriyor, projenin referansları nelerdir, yazılım nerede kullanılacak?

Ardından fonksiyonel gereksinimler (functional requirements) gelir. Hoca bunların üç biçimde verilebileceğini söyledi:

- Bütün yazılımın gereksinimlerini madde madde tek bir liste olarak sıralamak.
- Gereksinimleri modüllere ayırarak, her modül için ayrı liste vermek.
- Her modül için bir kullanım durumu diyagramı (use case diagram) ve onun belgesini, ya da yalnızca belgesini vermek.

Bu biçimlerin hiçbiri fonksiyonel olmayan gereksinimleri (non-functional requirements) içermez. Bu yüzden belgede fonksiyonel olmayan gereksinimlere ayrı bir bölüm ayrılması gerekir. Hoca bazı belgelerde bu bölümün eksik kaldığını gözlemledi.

> **Hocanın vurgusu:** Fonksiyonel gereksinimler hangi biçimde verilirse verilsin, fonksiyonel olmayan gereksinimler ayrı bir bölüm olarak mutlaka belgede yer almalıdır.

2. Bir Yazılımın Test Planı ve Test Raporunda Neler Olmalı?

Dönemin geri kalanındaki çalışma, SRS belgesi hazırlanan yazılım için bir yazılım test planı ve bir yazılım test raporu hazırlamaktır. Hoca bu iki belgede bulunması gerekenleri anlatırken, test planı ile test raporunun içeriğini somut olarak tarif etti.

Test edilecek yazılımın, testi yapanların kendi yazdığı yazılım olması gerekmez. Bir kitaptaki örnek proje, başka birinin projesi ya da daha önce yapılmış eski bir proje de seçilebilir. Hoca bunu ileride anlatacağı test bağımsızlığı ilkesiyle de ilişkilendirdi (bkz. Bölüm 4).

Test planı için hazır bir format verilmedi. Hoca test planının içeriğinin derste görüldüğünü ve bununla ilgili bir IEEE standardı bulunduğunu, planın oradan bakılarak oluşturulabileceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Hocanın adını vermediği bu standart büyük olasılıkla IEEE 829'dur (Software Test Documentation). Bu standart test planı, test senaryosu (test case) tanımı, test prosedürü ve test özet raporu gibi belgelerin şablonlarını tanımlar. Büyük olasılıkla 2013'ten sonra yerini ISO/IEC/IEEE 29119-3 standardına bırakmıştır.

Testlerde en az şu üç tür bulunmalıdır:

- **Birim testi (unit test):** Kod hangi dilde yazıldıysa o dilin birim test aracı kullanılır. Örneğin Java için JUnit. Hangi destek aracının (tool) kullanıldığı belirtilir.
- **Sistem testi (system test):** Açık kaynaklı bir test aracı araştırılır ve yazılıma uygulanır. Performans testi (performance test) ya da yük testi (load test) yapılabilir. Örneğin cevap süresine bakılabilir ya da aynı anda kaç kullanıcının bağlanıp işlem yapabildiği ölçülebilir. Güvenlik testi (security test) de seçilebilir.
- **Kullanıcı testi (user test):** Gereksinim belgesiyle karşılaştırılarak yapılır. SRS'deki hangi gereksinimin nasıl test edildiği belirtilir.

Her test için raporda şunlar yer alır:

- Test verisinin nasıl oluşturulduğu, yani nasıl seçildiği.
- Test verilerinin kendisi; kullanıcı testi için test senaryosu.
- Testin nasıl denendiği, nasıl çalıştırıldığı.
- Test ortamının nasıl bir ortam olduğu.
- Beklenen değer ile gerçekleşen değerin karşılaştırılması.

Hoca, testlerin yalnızca belgede anlatılmasını değil, en az bir testin çalıştırılarak gösterilmesini de istedi. Örneğin sistem testinin nasıl çalıştırıldığı ve hangi sonucun bulunduğu, ya da bir birim testinin çalıştırılması gösterilebilir.

3. Test Yönetimi ve Test Spesifikasyonu

Hoca bu bölümü bir tekrar dersi olarak hazırladığını, çünkü buradaki içeriğin doğrudan hazırlanacak test plan ve raporlarında kullanılacağını söyledi.

Test yönetimi (test management), test kapsamında yapılan bütün işlemleri ifade eder. Planlama, tasarım ve gerçekleştirme adımlarıyla yürütülür. Yazılım testlerini tanımlayan, planlayan, düzenleyen ve belgeleyen bütün belgeye ise **test spesifikasyonu** (test specification) denir. Hazırlanan test planı ve test raporu birlikte test spesifikasyon belgesini oluşturur. Test planı ve test senaryolarının hazırlanması bunun içindedir.

3.1 Test Planı Hangi Sorulara Cevap Verir?

Test planı şu soruları cevaplar:

- Hangi tip testler gerçekleştirilecek?
- Bunların zamanlaması nedir?
- Gideri (maliyeti) nedir?
- Hangi ortamlar kullanılacak?
- Hangi kaynaklar kullanılacak, yani hangi test için hangi araç (tool) kullanılacak?

Ardından test senaryoları (test case, test scenario) gelir. Saydam kutu testinde (white box test) hangi test verisinin nasıl seçildiği belirtilir. Birim testleri saydam kutu testi olarak gerçekleştirilir. Bütünlük testinde (integration test) testin hangi tipte yapıldığı ve hangi modüllerin test edildiği belirtilir. Test işlemleri sırasında hangi aracın nasıl kullanıldığı, beklenen sonucun ne olduğu ve gerçekleşen sonucun ne olduğu da yazılır.

3.2 Test Planının İçeriği

Hoca daha önce işlenen test planı içeriğini tekrar sıraladı:

- Test stratejisi.
- Test edilecek öğeler.
- Kullanılan doğrulama yöntemleri.
- Testlerin tamamlanma kriterleri: Hangi durumda testler tamamlanmış sayılır?
- Hata ve test sonuçlarının raporlanması.
- Testteki sorumluluklar: Kim neden sorumlu?
- Test ortamının nasıl oluşturulduğu.
- Test için başka eleman gerekiyorsa bunların eğitim ihtiyacı.
- Test takvimi: Proje plan takvimi gibi bir test takvimi.
- Test için risk yönetimi.
- Beklenen çıktılar, yani hangi sonuçların alınmasının beklendiği.

Risk yönetimi aslında yazılım proje yönetiminin konusudur. Fakat bütün yazılım geliştirme süreci içinde risklerin neler olduğu ve nasıl takip edildiği kaliteyi de ilgilendirir. Test yönetimi içinde testin kendine ait riskleri varsa ve bunlar nasıl yönetildiyse, test planında bunlardan da bahsedilebilir.

3.3 Test Senaryosunun İçeriği

Her test senaryosu için şunlar belirtilir:

- Senaryoyu kimin oluşturduğu.
- Senaryonun ilişkili olduğu gereksinim, yani hangi gereksinimi ya da kullanım durumunu (use case) karşıladığı.
- Gereksinimlerin amaçları.
- Testi gerçekleştirirken ön koşullar (precondition) veya varsayımlar.
- Testin girdileri.
- Test senaryosunun adımları.
- Beklenen sonuçlar.

3.4 Test Raporu

Test raporunda bu senaryolar kullanılarak testin nasıl gerçekleştirildiği anlatılır. Örneğin birim testinin nasıl çalıştırıldığı ve gerçek sonucun ne çıktığı yazılır. Beklenen sonuç ile gerçekleşen sonuç arasında bir karşılaştırma yapılabilir.

Test planı tek bir yazılımın bütünü için hazırlanır. Her test için ayrı plan yapılmaz (bkz. Örnek Sorular).

4. Test Ekibi, Bağımsızlık ve Optimum Test Miktarı

Test çalışmasında ekibin bağımsızlığı önemlidir. Yazılımı test etmekten sorumlu kişiler, onu tasarlayan ya da oluşturan gruptan olmazsa test daha başarılı ve daha tarafsız olur. Hoca, test edilecek yazılımın kendi yazdığınız proje olmak zorunda olmamasını bu ilkeye bağladı.

Test ekibinin görevleri şunlardır:

- Planlarda belirtilen testler için gerekli hazırlıkları yapmak.
- Testleri yürütmek.
- Testlerde bulunan hataları yazılımcıya bildirmek ve izlemek.
- Test sonuçlarını raporlamak.

Hoca burada bir ilke aktardı: “İyi bir testçi ne zaman mükemmele ulaşılacağını, iyinin yeterli olacağını bilir.” Mükemmele ulaşma koşulu yoktur. Hata sıfıra indirilemez, yalnızca azaltılabilir.

Bu, daha önce görülen **optimum test miktarı** kavramına bağlanır. Hoca şekli bu özet slayta koymadığını, ama daha önce görüldüğünü hatırlattı: Optimum test miktarı, hata sayısını en aza indirirken maliyeti de çok yukarı çıkarmayan test miktarıdır. Hoca bunu maliyet eğrisiyle hata bulma eğrisinin kesiştiği nokta olarak tarif etti.

> **Ders sonrası not:** Hoca optimum noktayı iki eğrinin kesişimi olarak tarif etti. Daha kesin ifadeyle optimum, toplam maliyetin (test maliyeti ile bulunamayan hataların maliyetinin toplamı) en düşük olduğu noktadır. Klasik grafiklerde bu nokta büyük olasılıkla iki eğrinin kesiştiği yerin yakınında gösterildiği için kesişim olarak anılır.

5. Teslimden Sonra Kalite: Yazılım Bakımı

Yazılım sisteminin geliştirilmesi, ürünün müşteriye teslimi ve kullanılmaya başlanmasıyla tamamlanır. Fakat kullanım sürecinde de bakım ve korumanın sürmesi gerekir. Buna bakım (maintenance) aşaması denir.

Yazılım bakımında yapılan işler üç gruptadır:

- **Hata düzeltme:** Test aşamasında bulunamamış, sonradan ortaya çıkan hataların düzeltilmesi.
- **İyileştirme:** Yazılımın yeteneklerinin geliştirilmesi. Örneğin bazı modüllerde performansı artırmak için yazılım ekibi ya da bakımı üstlenen ekip tarafından düzeltmeler yapılması.
- **Uyarlama:** Var olan uygulamalardaki değişikliklere ya da günün koşullarına yazılımın uyarlanması.

Hocaya göre genelde uyarlama tipindeki bakım işlemleri daha fazla görülür. Kurumun “bazı uygulamalarımızda değişiklik yaptık, bunu yazılıma uyarlayın” demesi ihtimali daha yüksektir.

Bakımda yapılan değişiklikler gelişigüzel yapılmaz. Gereksinim analizi, tasarım ve test adımlarıyla yeniden ele alınarak gerçekleştirilir.

6. Konfigürasyon Yönetimi

Yazılım kalitesini yürütmekte kullanılan bir diğer konu konfigürasyon yönetimidir (configuration management). Yazılım konfigürasyon yönetimi işlemlerinin uygulandığı her yazılım modülü bir **konfigürasyon maddesi** (configuration item) olarak tanımlanır.

Yazılım konfigürasyon yönetimi etkinlikleri, her konfigürasyon maddesi için dört aşamada işlem yapılmasıdır:

İşlem	Açıklama
Tanımlama	Her maddenin önce tanımı yapılır, genellikle bir veri tabanı uygulaması şeklinde
Değişiklik kontrolü	Maddede yapılan değişikliklerin kontrolü
Denetim	Maddelerin denetimi
Raporlama	Durumun raporlanması

Bunun yanında yazılım konfigürasyonunu oluşturan maddeler, yani **yazılım konfigürasyon elemanları** şunlardır: programlar, belgeler, veri yapıları ve test verisi. Hoca bunun daha önce sınavda da sorulduğunu hatırlattı (bkz. Örnek Sorular). Konfigürasyon yönetiminin nasıl yürütüleceğini belirlemek için standartlara göre çalışmak gerekir.

7. Yazılım Risk Yönetimi

Hoca, test planında risk yönetimine de yer verilebileceğini söyledikten sonra, risk yönetiminin nasıl kurulduğunu ve riskin ne olduğunu sordu.

7.1 Risk Nedir?

Risk olasılıkla ilişkilidir. Bir problemin ortaya çıkma ihtimali vardır, ama çıkabilir de çıkmayabilir de. Bir şey %100 olasılıkla ortaya çıkıyorsa artık risk değildir; o anda var olan bir problemdir. Risk gelecekte ortaya çıkabilecek bir şeydir. Yani risk hem gelecekle ve olasılıkla hem de kayıpla ilişkilidir. Risk ortaya çıktığında istenmeyen sonuçlar ve kayıplar doğurur.

Riskle uğraşmak için iki tür çözüm vardır:

- **Reaktif (reactive) çözüm:** Problem ortaya çıkınca onu bertaraf etmek için işlem yapmak.
- **Proaktif (proactive) çözüm:** Problem gerçekleşmeden onu önlemeye çalışmak.

Bir riskin önemi belirlenirken iki soru sorulur: Bu riskin ortaya çıkma olasılığı nedir? Ortaya çıktığında ne kadar kayıp olacak, bizi ne kadar etkileyecek?

7.2 Risk Türleri

Hoca proje riskleri, teknik riskler ve iş (işletmecilik) riskleri gibi risk çeşitlerinden söz etti. Pressman'ın kitabında daha kapsamlı bir sınıflandırma bulunduğunu da ekledi: müşteri riski, işletmecilik riski, teknik riskler, geliştirme ortamı riskleri, proje riskleri gibi çok sayıda sınıf.

- **Proje riskleri:** Proje planını tehdit eder. Bütçeyle, zamanla, personelle ve çalışanlarla ilişkili risklerdir.
- **Teknik riskler:** Üretilen yazılımın kalitesini ve zamanında bitirilmesini etkiler; yazılımın gerçeklenmesini zorlaştırır. Analiz, tasarım ve bakım aşamasında teknolojik sorunlar olabilir. Örneğin kullanmaya karar verdiğimiz teknolojinin sürekliliği var mı? Teknik riskler zamanında bitirmeyi etkilediği için proje risklerini de etkileyebilir.
- **İşletmecilik (iş) riskleri:** Pazar riski, ürüne talep olup olmayacağı. Ürünü geliştiriyoruz ama onu satacak elemanımız ya da bir satış stratejimiz var mı?

7.3 Risk Tablosu

Riskleri değerlendirmek için bir risk tablosu oluşturulur. Hoca slaytta tablonun şeklinin olmadığını, ama yapısını anlattı. Tablonun her satırı bir riske ayrılır. Sütunlar şunlardır:

Sütun	İçerik
Risk tanımı	Riskin açıklaması (birinci sütun)
Sınıf	Riskin dahil olduğu sınıf: işletmecilik, personel, teknik vb.
Olasılık	Riskin olma olasılığı: %90, %50, %40 gibi
Etki	Risk gerçekleşirse kayıp: yüksek, orta, düşük ya da sayısal bir aralık
Önem	Olasılık × etki

Tablonun amacı, hangi riskleri yakın takibe alacağımıza karar vermektir. Riskin önemi, olasılık ile riskin proje üzerindeki kayıp etkisinin çarpımıdır:

$$\text{Önem} = \text{Olasılık} \times \text{Etki}$$

Önem değerine göre riskler yüksek önemli, orta önemli gibi sınıflanır. Önemi fazla olan riskler ayrılır ve her kilometre taşında (milestone) yeniden gözden geçirilir: Olasılığı hâlâ ne kadar? Olduğunda etkisi yine aynı mı?

> **Ek bilgi (derste anlatılmadı):** Literatürde olasılık ile kaybın çarpımı risk maruziyeti (risk exposure) olarak geçer ve $RE = P \times C$ biçiminde yazılır. Burada P riskin gerçekleşme olasılığı, C gerçekleşmesi durumundaki maliyettir. Bu tanım büyük olasılıkla Boehm'in risk yönetimi çalışmalarına dayanır ve Pressman'ın kitabında da yer alır.

7.4 Önlem ve Risk Takibi

Önemli riskler için önlem belirlenir: Ne yaparsak bu riski azaltabiliriz? Hocanın örneği, yazılım geliştiricilerin başındaki yazılım mühendisinin işten ayrılma olasılığıydı. Bu önemli bir risktir. Önlem olarak yanına bir yardımcı mı alınacak, yoksa bir anlaşma mı imzalanacak? Her önlemin bir de maliyeti vardır. Kararlar alınacak önleme ve onun maliyetine göre verilir ve takip edilir.

Bu yüzden risk takibi önemlidir. Ele alınan her risk için ayrıca bir tablo tutulur. Her kilometre taşında şu değerlendirmeler yapılır: Olasılığı değişti mi, etkisi farklılaştı mı, önlemi alındı mı, maliyeti ne oldu?

8. Yeniden Kullanılabilirliği Artıran Yaklaşımlar

Hoca yazılım kalite ölçütlerindeki ölçüm konusuna bu kez girmeyeceğini, çünkü kalite ölçütleri, McCall modeli ve nesneye yönelik ölçütler üzerinde daha önce durulduğunu söyledi. Kalite iyileştirme de bir önceki haftada işlenmişti.

Yeniden kullanılabilirlik (reusability) yazılım kalite özelliklerinden biridir. Hoca bu özelliği artıran yaklaşımları sıraladı:

- Nesneye yönelik programlama (object-oriented programming).
- Bileşen tabanlı yazılım geliştirme (component-based development).
- Tasarım kalıpları (design patterns).
- API'ler, program kütüphaneleri (libraries) kullanmak.
- Kod üretimi (code generation).
- Cephe yönelimli yaklaşım (aspect-oriented): Yazılıma farklı bakış açılarından bakarak, örneğin gereksinim analizini farklı açılardan değerlendirerek daha iyi bir analiz yapmayı hedefler.
- Çerçevelerin (frameworks) kullanımı.
- Servis yönelimli mimari (service-oriented architecture).
- Yazılım ürün hatları (software product lines) ve yazılım ürün aileleri.

9. Sürüm Yönetimi (Release Management)

Aradan sonra hoca sürüm yönetimine geçti ve bunun konfigürasyon yönetimiyle ilişkili bir konu olduğunu söyledi.

9.1 Tanım ve Verilen Kararlar

Yeni bir yazılım geliştirildiğinde kullanıma alma aşamasında sürüm yönetimi yapılır. Sürüm yönetimi, bir ürünün, kurumdaki yazılım sisteminin ya da servisin işletmenin ihtiyaçlarını karşılayacak biçimde ilk kez kullanıma alınması ya da yenilenmesidir.

Son kullanıcıların kullanacağı ürünün kullanıma alınmasında hangi işlemlerin yapılacağı ve hangi kuralların geçerli olacağı sürüm yönetimiyle belirlenir. Bu kararlara örnekler:

- Yayınlama işlemi hangi gün olacak, hangi saatte yapılması uygun olur?
- Kullanıcılar eski ürünü kullanmaktan yeni ürüne hangi şekilde geçecek?
- Uygulamaya konacak yeni sistem, yaşayan (canlı) uygulama ortamıyla nasıl bütünleşecek ve yaşayan uygulama nasıl koruma altına alınacak?

Bazı kurumlarda sürüm yönetimini teknik geliştirme ekibi yürütür. Bazı organizasyonlarda ise işletmecilik ve iş kurallarına uyumu sağlayacak başka bir grup yürütür.

9.2 ITIL Çerçevesi ve İki Tarafın Ortak Kararı

Sürüm yönetimi ve kullanıma alma işlemi IT Infrastructure Library (ITIL) çerçevesi (framework) tarafından belirlenir. Birçok bilgi teknolojisi işleminde olduğu gibi sürüm yönetiminde de bu çerçevedeki yapı izlenir. ITIL, sürüm ve kurulum yönetiminin tüm uygulamaların yönetimi ve stratejinin belirlendiği ekip tarafından planlanmasını ve iş listesinin buna göre yapılmasını tanımlar.

Sürüm kavramı, yazılım geliştirme ortamı ile işletmecilik (organizasyon) ortamının birleşimidir; ikisinin ortak kararlarıyla yürütülür. Sürüm yönetimi, kullanıcılar ile yazılım geliştirme proje ekibi arasında bir köprü rolü üstlenmelidir. Bu yüzden sürüm yönetiminin nasıl gerçekleşeceğine iki taraf birlikte karar vermelidir.

Bir kurumda aynı anda yürüyen birçok proje varsa, bunların birbiriyle ilişkisi de kararlarda önem taşır. Önce hangi uygulama devreye alınmalı? Ona bağlı diğer uygulama ne zaman devreye alınacak? Uygulamalar arasında kaynak ve bilgi transferi gerekebilir; bu da sıralamayı etkiler.

Özetle ITIL çerçevesi, sürüm yönetiminde hizmete alma, kurulum, rol tanımları ve sıralama konusunda yol göstericidir.

> **Hocanın vurgusu:** Sürüm yönetimi yalnızca projeyi geliştiren ekibin işi değildir. İşletmecilik süreçlerini yönetenlerin bilgisini de gerektirir. İki tarafın birlikte hareket etmesi gerekir.

9.3 ITIL Sürüm Yönetimi Süreci: Altı Alt Süreç

ITIL, sürüm yönetimi için altı alt süreç tanımlar:

Alt süreç	Açıklama
Sürüm yönetimi desteği	Süreç boyunca bütün adımlara destek veren, rehberler oluşturan baş adım
Sürüm planlama	Sürümün kapsamı, izlenecek politika, paketlerin hangi sırayla işleme gireceği
Sürümün oluşturulması	Yenileme ya da yeni ürün tamamlanır; geçerleme işlemlerinin tüm sürüm bileşenlerinde yapıldığı denetlenir
Sürüm dağıtımı (release deployment)	Yeni sürümün uygulamaya konması; gerçek üretim ortamına geçiş, son kullanıcıların eğitimi ve eğitim belgelerinin aktarımı
Sürüm sonrası destek	Yeni sürüm kullanıma alındıktan sonraki dönem; belli bir süre koruma ve destek altında, çıkan hata hemen düzeltilir
Sürüm kapatma	Sürüm etkinliklerinin formal kapanışı; belgelerin doğrulanması, çıktıların raporlanması, proje ekibine geri besleme

Dağıtım adımı, yazılımın geliştirme ekibinin elinden çıkıp son kullanıcılara aktarıldığı adımdır. Sürüm sonrası destekte kurulum, uygulamaya koyma ve sürüm bir ekip çalışması şeklinde yürür; çıkan hatalar takibe alınır.

> **Ek bilgi (derste anlatılmadı):** Hoca alt süreçlerin İngilizce adlarını yalnızca kısmen söyledi. ITIL v3'te bu alt süreçler büyük olasılıkla şu adlarla geçer: Release Management Support, Release Planning, Release Build and Test, Release Deployment, Early Life Support ve Release Closure.

9.4 Proje Yönetimiyle Benzerlik: Kapatma ve Arşiv

Hoca kapatma adımını proje yönetimine benzetti. Proje yönetiminde de adımlar projenin başlatılması, planının oluşturulması, yürütülmesi ve kapatılmasıdır. Kapatma aşamasında projenin bittiğine dair tanımlar yapılır ve bir arşiv oluşturulur. Arşive şu bilgiler aktarılır: Proje ne kadar sürdü, ne kadar iş gücü harcandı, maliyeti ne oldu, hangi riskler çıktı ve nasıl kapatıldı, hangi teknolojiler kullanıldı? Bu geliştirme bilgisi, sonraki projelerde tahmin yürütebilmek için bir arka plan oluşturur. Aynı şekilde sürüm kapatma aşamasında da sürümle ilgili bilgiler kaydedilir.

9.5 Sürüm Yöneticisi (Release Manager)

Sürüm yöneticisi, sürüm yönetiminin planlamasını ve koordinasyonunu sağlayan kişidir. Yeni bir sürümün kullanıma alınmasının etkin ve güvenli biçimde tamamlanmasını sağlar. Tipik olarak bir bilgi teknolojileri uzmanı olması beklenir. Sürüm etkinliklerini koordine edebilmek için yazılım geliştirme standartları, IT süreçleri ve bütün iş süreçleri hakkındaki standartlar, süreçler ve araçlar konusunda deneyim sahibi olmalıdır.

Sürüm yöneticisinin özellikleri:

- Hem IT bilgisine sahip olmalı hem de işletmecilik tarafındaki liderlerle çalışabilmelidir.
- IT proje ekibi ile işletmecilik tarafındaki görevlilerin bir arada çalışmasını sağlamalıdır. Hoca bunu "iyi bir orkestra şefi gibi çalışması" diye tarif etti.
- Ürün yönetimi kapsamında pazarlama, araştırma, geliştirme ve diğer ekiplerle koordineli çalışmalıdır.
- Büyük uygulamaların sürümlerini yürüten ekip birden fazla uygulamayı da yürütüyor olabilir. Bu durumda diğer projeler ve alt projeler hakkında da bilgi sahibi olmalıdır.
- Kaliteden de sorumludur ve kurumun kalite grubuyla yakın çalışmalıdır. Kurumun standartlarına ve yol haritasına uygun hareket etmelidir.

9.6 Konfigürasyon Yönetimiyle İlişkisi

Sürüm yönetimi, kullanıma alınan ürün hakkındaki bilgilerle çalışır: ürünün teknik alt elemanları, kullanılan test verisi, kullanılan veri tabanı gibi. Bütün bu bilgiler konfigürasyon yönetimi içinde kontrol edildiği için, sürüm sırasında yapılan işlemler de konfigürasyon yönetimine aktarılır. Hoca, sürüm yönetiminin adımlarını ve konfigürasyon yönetimine nerede bilgi aktarıldığını şematik olarak gösteren şekillerin internette ve başka kaynaklarda bulunabileceğini söyledi.

9.7 Sürüm Yönetiminin Faydaları ve DevOps

Sürüm yönetiminin faydaları:

- Yeni değişiklikler ve özellikler kullanıcılara daha hızlı dağıtılır. Bunu yaparken kullanıcıların aldığı risk de azalır.
- İş ortamına aktarma, kurulum ve kullanıma alma için iş planları önceden tahmin edilebilir ve iyi planlanmış olarak yürütülür.
- Yeni bir servis (bu bir yazılım ya da ürün olmayabilir, bir hizmet de olabilir) gereksinimleri karşılayacak biçimde uygulamaya alınır.
- Hem kullanıcılara hem destek personeline uygun bilgi transferi sağlanır.

DevOps, Development (geliştirme) ve Operations (operasyon) kelimelerinin birleşimidir. DevOps stratejisi, geliştirme ve operasyon işlerinin aynı mantıkla ve aynı standartla yürütülmesidir. Özellikle çevik (agile) yazılım geliştirme yaklaşımında daha çok kullanılır. DevOps stratejisi kullanılıyorsa sürüm yönetimi bununla yakından ilişkilidir. Hocaya göre birçok IT departmanı DevOps yaklaşımında etkin bir sürüm yönetimini benimsemiştir ve piyasada DevOps stratejilerini destekleyen birçok ürün bulunmaktadır. Hoca, çevik projelerde sürüm yönetiminin nasıl yürüdüğü ve DevOps ile ilişkisi hakkında daha geniş kaynaklara bakılmasını önerdi.

> **Ders sonrası not:** Altyazıda "Ecel" diye geçen kelime "agile" (çevik) kelimesidir.

10. Mobil Uygulamalarda Kalite ve Test

Son bölümde hoca, artık birçok yazılımın geliştirildiği özel bir uygulama alanına, mobil uygulamalara geçti. Bu alanda testin ve kalitenin nasıl sağlandığını, geçen yılın yüksek lisans öğrencilerinden birinin hazırladığı bir sunum üzerinden anlattı.

10.1 Neden Ayrı Bir Bakış Gerekir?

Hoca bir benzetme yaptı: Nesneye yönelik yazılımlar için ölçütlerden bahsederken, yapısal yaklaşımdan farklı olarak nesne yaklaşımında hangi özellikler öne çıkıyorsa, nesneye yönelik

ölçütlerde onların ölçümü öne çıkıyordu. Örneğin CK metriklerinin hepsi nesneye dayalı özellikleri ölçen birimlerdi. Aynı şekilde mobil cihazlarda kullanılan yazılımlarda hangi özellikler önemliyse, kalite ve test aşamasında o özellikleri test eden taraflar ağır basmalıdır.

Mobil cihazların rolü büyük olduğu için bunların güvenilirliği ve uyumluluğu daha fazla önem taşır. Mobil uygulamalarda iki süreç vardır: yazılım hazırlanırken yapılan test ve kullanıma alındıktan sonra uygulamanın ortama nasıl uyum sağladığını değerlendiren adaptasyon süreci.

Mobil uygulama testi, diğer uygulamalara göre daha pahalı ve zaman alıcıdır. Asıl önemli olan, uygulamanın kullanılabilirliği, mobilitesi ve güvenliği sağlayıp sağlamadığına bakmaktır. Bir cep telefonunu test etmenin zorluğu maliyetidir. Bu maliyet emülatörlerle azaltılır, ama emülatörlerin güvenilirliği çok yüksek değildir ve hata payı kalır. Zaman zaman kapsamlı bir test yapılmadan, yalnızca yazılım işlevlerine odaklanıp emülatörde test edilerek sürüm aşamasına gelinir. Minimum testle hızlıca yapılan böyle bir çalışma sonradan kullanımda risk oluşturur. Yeni bir sürüme geçilirken mobil test başarısı önemlidir.

10.2 Mobil Uygulama Türleri ve GUI

Mobil uygulamalar akıllı telefonlarda ve tabletlerde belirli görevleri yerine getirmek için çalışan uygulamalardır. Hoca bu uygulamalarda grafik kullanıcı arayüzünün (GUI) önemli olduğunu, uygulamanın farklı kullanıcı arayüzlerinde nasıl çalıştığına bakmak gerektiğini ve en fazla hatanın burada ortaya çıktığını söyledi.

Uygulama türleri: mobil web uygulamaları, yerel (native) mobil uygulamalar ve hibrit (hybrid) uygulamalar. Hoca mobil uygulamaların genellikle sunucu taraflı (server side) teknolojiyle oluşturulduğunu ve internet bağlantısına ihtiyaç duyduğunu söyledi.

> **Ders sonrası not:** Hoca “mobil uygulamalar genellikle server side teknolojisiyle oluşturulur” dedi. Bu ifade büyük olasılıkla mobil web uygulamaları için geçerlidir. Yerel (native) uygulamalar cihaza kurulur ve platformun kendi geliştirme araçlarıyla yazılır; hibrit uygulamalar ikisini birleştirir.

10.3 Başarılı Bir Mobil Uygulamanın Ayırt Edici Özellikleri

Hoca nesneye yönelik örneğe bağlayarak mobil uygulamaların öne çıkan, ayırt edici özelliklerini sıraladı:

- Bağlantı (connectivity).
- Uygunluk ve kaliteli tasarım.
- Desteklenen cihazlar.
- Yeni bir programlama dili kullanıldıysa onun özellikleri.
- İçerik bilgisi ve güvenlik.
- Kişiselleştirme kolaylığı.
- Ulaşılabilirlik (accessibility).
- Yerelleştirme (localization).

Mobil uygulamalar neredeyse her zaman internet bağlantısına ihtiyaç duyar. İnternet şebekesi hız, güvenilirlik ve güvenlik açısından değişebilir. Bu yüzden farklı ağlarda ve farklı bağlantı senaryolarında işlevsel testler yapılmalıdır. Grafik kullanıcı arayüzü ve uygulamanın kullanım kolaylığı farklı cihazlarda test edilmelidir.

Mobil uygulama testi, mobil uygulamalara ait etkinliklerin iyi tanımlanmış yazılım test yöntemleri ve araçları kullanılarak mobil cihazlarda test edilmesidir.

10.4 Öne Çıkan Test Alanları

Kullanılabilirlik (usability), birlikte çalışabilirlik (interoperability), mobil bağlantı ve güvenlik test açısından büyük önem taşır.

Mobil test süreci şu nedenlerle karmaşıktır: Cep telefonu üreticileri tarafında yüksek bir dinamiklik vardır, yazılım sık sık güncellenir ve her uygulamanın kendine özgü iş ve veri akışı vardır. Bu yüzden birleşik, destekleyici test işlemlerini tam olarak yapmak her zaman mümkün olmaz. Mobil uygulamalar donanım uyumluluğu, yazılım güvenilirliği ve uygulama işlevselliği gibi farklı amaçlar için farklı yaklaşımlarla test edilir.

Hocanın değindiği mobil test yaklaşımları:

- **Birlikte çalışabilirlik:** Uygulamanın diğer uygulamalarla uyumlu olduğundan ve çapraz kullanım işlevselliği sağladığından emin olmak için aynı uygulama farklı platformlarda test edilir.
- **Mobil sistem güvenlik testi:** Mobil cihazlarda kişisel ve ticari olarak depolanan bilgilerin gizliliği için hassas verilerde şifreleme ve şifre çözme teknikleri kullanılır. Saldırılar SMS, MMS, Wi-Fi ve Bluetooth gibi iletişim araçlarından ya da web tarayıcısı ve işletim sistemindeki yazılım açıklarından gelebilir. Güvenlik testinde bunların hepsi göz önünde bulundurulmalıdır.
- **Yapısal test:** Uygulama saydam kutu (white box) testiyle de test edilir.
- **Mobil teknolojiye özgü yapılar:** Mobil uygulamalar, mobil dillere ve yeni mobil teknolojilere uyum, algılama ve enerji tüketimini yönetmek için bazı özel yapılar ekler. Yeni kapsama (coverage) kriterleri en azından mobilite algılamasını ve enerji yapılarını göz önünde bulundurmalıdır.
- **Otomatik tarama testi:** Otomatik tarama test yazılımları, hataların ve cihaz tutarsızlıklarının hızlıca bulunmasını sağlayan bir araç olarak kabul edilir.

10.5 Mobil Test Süreci ve Test Türleri

Test sürecimiz daha önce test stratejisinde çok tekrar edildi: birim testi, bütünlük testi, sistem testi, regresyon testi (regression test), kabul testi (acceptance test). Mobilde bunlara kullanıcı arayüzü testi ayrı bir test olarak eklenir, çünkü çok önem taşır. Kullanıcı arayüzü testi, sistem testi tamamlandığında ya da onunla paralel olarak sürece eklenir.

Uygulamanın test edilmesinde fonksiyonel testler ve durum yaklaşımlı test kullanılır. Durum yaklaşımlı testte uygulama farklı durumlar ve modlar için modellenir. Örneğin düşük pil durumunda, toplantı modunda ya da uçuş modunda uygulama nasıl çalışacak? Hem uygulamanın hem de çalışacağı ortamın belirlenmesi ve farklı durumlar için test edilmesi gerekir.

Hocanın saydığı test türleri:

- Fonksiyonel test (kara kutu testleri).
- Performans testi.
- Güvenlik testi (security test).
- Kullanılabilirlik testi (usability test).
- Yük ve stres testi (load/stress test).
- Yerelleştirme testi (localization test).
- Senkronizasyon testi (synchronization test).

10.6 Kullanıcı Arayüzü, Performans ve Güvenlik Testi

Kullanıcı arayüzü testi: Farklı cihazların yeterli veriyi sağlayıp sağlamadığı ve uygulamanın farklı cihazlarda doğru görüntülenip görüntülenmediği test edilir. Farklı ekran tiplerinde, markalarda, modellerde ve kullanıcı etkileşiminde yakalanan farklı girdilerde uygulama çalıştırılmalıdır. GUI testini hızlandırmak için bu işleri otomatik yapan araçlar vardır; her grafik arayüz için ayrı araçlar geliştirilebilir.

Performans testi: Uygulamanın başarılı biçimde kullanıma alınmasında önemlidir. Uygulamanın hızını ve kullanım kolaylığını etkileyen etkenlerden biri mobil taşıyıcılardır (operatörler). Uygulama performansı son kullanıcı için zayıfsa kullanıcı diğer uygulamalara ve hizmetlere yönelebilir. Bölgeye bağlı olarak farklı cihazlar ve taşıyıcılarla test yapılmalıdır. Mobil uygulamaların performansı ve güvenilirliği cihazın donanım gücüne, çalışma moduna, bağlantı kalitesine ve değişkenliğine bağlıdır.

Güvenlik testi: Mobil ağların farklı güvenlik seviyeleri vardır. Virüsler kişisel bilgilere erişebilir. Şifrelerin ya da kullanıcı adlarının cihazda şifrelenmeden bırakılması, bunların başkaları tarafından kullanılabilmesi tehlikesini doğurur. Uygulama cihaza yüklenirken yürütme izinleriyle ilgili seçenekler çıkar. Örneğin uygulamanın cihazdaki bağlantı türlerine, etkileşimlere ya da kullanıcı verilerine erişmesine izin vermeniz ya da reddetmeniz istenebilir. Kurulumdan sonra bile cihazın güvenlik ayarları ve uygulama izinleri değiştirilebilir. Test senaryoları (test case) bu durumların hepsini kapsamalıdır.

10.7 Cihaz Çeşitliliği ve Test Seçenekleri

Cep telefonlarında çok farklı özellikler ve farklı donanım bileşenleri vardır. Birçok cihazda test yapmak maliyetli ve zaman alıcıdır. Bu yüzden üç seçenekten biri kullanılır:

Seçenek	Artı	Eksi
Yalnızca gerçek cihazlar	Daha sağlıklı sonuç	Maliyetli
Yalnızca emülatörler	Ucuz	Sınırlamalar, farklı sonuçlar elde edilebilmesi
İkisinin kombinasyonu	İki yaklaşımın birlikte kullanımı	Altyazıda ayrıca belirtilmedi

Sunumdaki bilgiye göre testlerin %30-40'ı emülatörler kullanılarak yapılıyor. Bir de mobil bulut bilişim çözümleri vardır: Bulut yaklaşımında mobil cihaza bir web arayüzü üzerinden erişilir, uygulamalar test edilir ve yönetilir. Gerçek ağlara bağlı gerçek zamanlı cihazlar kullanılır. Bu çözümlerle zorlukların bir kısmı aşılabilir.

10.8 Mobil Testin Zorlukları

Hoca zorlukları sorularla özetledi:

- Uygulama hangi mobil cihazları ve işletim sistemi sürümlerini destekleyecek?
- Uygulamaların bu platformlarda çalıştığından emin olmak için nasıl test ederiz?
- Platformlar arasındaki farklara uyum sağlamak için hangi değişiklikleri yapabiliriz?
- Yeni cihazlar, teknolojiler ve uygulamalar çok sık güncellendiğine göre testler bunlara nasıl yetişecek?
- Testin yeterli olduğu nasıl anlaşılacak?

Mobil uygulama test matrisi oluşturulduğunda, değişen her faktör (güvenlik, araçlar, bağlı olunan ağ vb.) matrisi etkileyerek karmaşık bir senaryo ortaya çıkarır. Başlıca zorluklar:

- **Parçalanma (fragmentation):** Çeşitli işletim sistemlerinde çalışan çok sayıda mobil cihaz, farklı kullanıcı arayüzleri ve uyumluluk sorunları. Bunlar zorluk seviyesini artırır ve testi parçalar.
- **Ağ:** Her mobil operatör farklı destek verebilir. Bu yüzden bütün ağlarda test etmek gerekir.
- **Güvenlik:** Kişisel bilgilerin korunduğundan emin olunmalı ve bu taraf da test edilmelidir.
- **Test araçlarının çeşitliliği:** Mobil uygulamaları test eden çok sayıda açık kaynaklı test aracı vardır; bunların gözden geçirilmesi gerekir.

Hoca, sistem testleri için araç ararken bu konulara bakılmasını önerdi. Hoca derste belirli bir mobil test aracının adını vermedi.

> **Ek bilgi (derste anlatılmadı):** Mobil test için yaygın açık kaynaklı araçlar arasında büyük olasılıkla Appium (platformlar arası UI otomasyonu), Espresso (Android) ve XCUITest (iOS) sayılabilir. Android tarafında otomatik "tarama" (crawling) testine büyük olasılıkla Monkey ve benzeri araçlar örnek verilebilir.

Örnek Sorular

Soru 1 (derste sözlü, 00:51:54): Test planı her test için ayrı ayrı mı hazırlanır?

Cevap (derste verildi): Hayır. Genel olarak bir yazılım test edilir ve bu yazılımın bütünü için tek bir test planı oluşturulur. Her test için ayrı plan yapılmaz.

Soru 2 (derste sözlü, 00:40:30; daha önce sınavda sorulmuş): Yazılım konfigürasyon elemanları nelerdir?

Cevap (derste verildi): Programlar, belgeler, veri yapıları ve test verisi. Bunların hepsi yazılım konfigürasyonunu oluşturan maddelerdir. Konfigürasyon yönetiminde bu maddeler üzerinde yapılan işler ise tanımlama, değişiklik kontrolü, denetim ve raporlamadır.

Soru 3 (derste sözlü, 00:41:41): Risk nedir ve bir riskin önemi nasıl belirlenir?

Cevap (derste verildi): Risk, gelecekte ortaya çıkabilecek, olasılık ve kayıpla ilişkili bir problemdir; %100 olasılıkla ortaya çıkan bir şey risk değil, var olan bir problemdir. Riskin önemi, ortaya çıkma olasılığı ile ortaya çıktığında yaratacağı kayıp etkisinin çarpımıdır: Önem $=$ Olasılık $\times$ Etki. Önemi yüksek riskler her kilometre taşında yeniden gözden geçirilir.

Soru 4 (derste sözlü, 01:50:09): Mobil uygulamaların test süreci neden karmaşıktır?

Cevap (derste verildi): Cep telefonu üreticileri tarafında yüksek dinamiklik vardır, yazılım sık güncellenir ve her uygulamanın kendine özgü iş ve veri akışı vardır. Bu yüzden birleşik test işlemlerini tam olarak yapmak her zaman mümkün olmaz. Buna cihaz ve işletim sistemi çeşitliliği (fragmentation), operatörlere göre değişen ağlar, güvenlik ve test aracı çeşitliliği eklenir.

Soru 5 (derste sözlü, 02:05:59): Testlerde kapsama (coverage) beklentisi var mı?

Cevap (derste verildi): Var; en azından bazı testler için yapılabilirse iyi olur. Fakat bu konuda yapılan ilk çalışma olduğu için temel şeylerin yapılması, yapılabildiği kadarının yapılması bekleniyor.

Dersin Toparlaması

Bu derste önce iyi bir gereksinim belgesinin yapısı ve fonksiyonel olmayan gereksinimlerin ayrı bir bölüm olarak verilmesi gerektiği hatırlatıldı. Ardından test spesifikasyonu tekrar edildi: test planının cevapladığı sorular ve içeriği, test senaryosunun bileşenleri ve test raporunda beklenen ile gerçekleşen sonucun karşılaştırılması. Test ekibinin bağımsızlığı, hatanın sıfırlanamayacağı ve optimum test miktarı vurgulandı. Teslimden sonraki kalite için yazılım bakımının üç türü ve konfigürasyon yönetiminin dört işlemi ele alındı. Risk yönetiminde riskin tanımı, risk türleri, risk tablosu ve önem = olasılık × etki ilişkisi anlatıldı.

Dersin ikinci yarısında sürüm yönetimi ITIL çerçevesindeki altı alt süreç, sürüm yöneticisinin rolü, konfigürasyon yönetimiyle ilişkisi ve DevOps bağlantısıyla işlendi. Son olarak mobil uygulamalarda kalite ve test ele alındı: ayırt edici özellikler, öne çıkan test alanları (kullanılabilirlik, birlikte çalışabilirlik, bağlantı, güvenlik, GUI), gerçek cihaz ve emülatör seçenekleri ve mobil testin zorlukları. Hoca bir sonraki hafta yeni bir konu anlatacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.